CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

GMX v2 (ExchangeRouter, RoleStore, DataStore, Oracle)

0x69C527fC77291722b52649E45c838e41be8Bf5d5

Arbitrum

Compiler: v0.8.18+commit.87f61d96

Date: October 22, 2026

OVERALL RISK RATING

High

Manual review of verified source. A risk snapshot, not a guarantee.

Executive Summary

Independent review of GMX v2 on Arbitrum, covering the ExchangeRouter at 0x69C527fC77291722b52649E45c838e41be8Bf5d5, the RoleStore at 0x3c3d99FD298f679DBC2CEcd132b4eC4d0F5e6e72, the DataStore at 0xFD70de6b91282D8017aA4E741e9Ae325CAb992d8 and the Oracle reached from them. 120 verified source files, 24,947 lines, compiled with solc 0.8.18. Role membership was additionally read from the live contracts on 22 October 2026 rather than assumed, and the counts below are what the chain returned.

Perpetuals are the hardest contract class in DeFi and the order, position and ADL logic here is the work of a team that knows it. No flaw was found in the order lifecycle, the position accounting or the auto deleverage path. Two design decisions deserve to be named as correct rather than merely absent from the findings. Arbitrary external calls, the single most common way a router is turned into a drain, are executed by a separate ExternalHandler so that the router's own token allowances are unreachable from them, which is the opposite of the pattern that made a five million dollar router a High finding in an earlier review in this repository. And auto deleveraging is gated on both a per market flag and an oracle timestamp no older than the moment that flag was set, so a keeper cannot deleverage on a stale price.

The risk is concentrated in one place: the price that reaches the engine, and what bounds it. GMX v2 prices positions from keeper supplied data, and the only independent check on that data is a deviation comparison against a Chainlink feed. That comparison is conditional on a DataStore address being non-zero, and both of its bounds are plain DataStore integers. Three single slot writes, by any of the forty three addresses currently holding CONTROLLER, disable or widen the only thing standing between a keeper price and every open position. DataStore emits no event on any write, and RoleStore emits no event on any role change, so none of it is visible to an off chain monitor.

This is a centralization and observability rating, not a claim of an exploit, and it is worth saying plainly that the authority above all of this is better than most. ROLE_ADMIN is held by three timelock contracts, not by a key. That is the right answer to the question this repository asks. The findings below are about what is not bounded in the contracts themselves, and about what an outside observer cannot see.

Findings (6)

GMX-01 · No SWC mapping (oracle governance)

High

The only independent check on keeper supplied prices is disabled by a single storage write

DESCRIPTION

GMX v2 prices every position from data supplied by an oracle provider. Two checks are applied to that data. The first is staleness, which rejects a price older than MAX_ORACLE_PRICE_AGE. The second, and the only one that constrains the value rather than the age, is a deviation comparison against a Chainlink reference price.

That second check is conditional:

(bool hasRefPrice, uint256 refPrice) = ChainlinkPriceFeedUtils.getPriceFeedPrice(dataStore, token);
if (hasRefPrice) {
_validateRefPrice(token, validatedPrice.min, refPrice, maxRefPriceDeviationFactor);
_validateRefPrice(token, validatedPrice.max, refPrice, maxRefPriceDeviationFactor);
}

And hasRefPrice is decided entirely by one storage slot, at ChainlinkPriceFeedUtils.sol:17-21:

address priceFeedAddress = dataStore.getAddress(Keys.priceFeedKey(token));
if (priceFeedAddress == address(0)) {
return (false, 0);
}

Writing the zero address to priceFeedKey(token) does not cause a revert, does not log anything, and does not fail any other check. It silently removes the deviation comparison for that token, after which a keeper price is accepted on staleness alone.

The bounds of the check are equally mutable. Both are plain integers read from the same store, at Oracle.sol:249-250:

uint256 maxPriceAge = dataStore.getUint(Keys.MAX_ORACLE_PRICE_AGE);
uint256 maxRefPriceDeviationFactor =
dataStore.getUint(Keys.MAX_ORACLE_REF_PRICE_DEVIATION_FACTOR);

So there are three independent single slot writes that each weaken or remove the constraint: zero the feed address for a token, raise the deviation factor until any price passes, or raise the maximum price age. None of the three is bounded in the contracts reviewed, and nothing distinguishes a token that has no reference feed by design from one whose feed was removed.

IMPACT

Whoever can write these DataStore keys can cause the engine to accept an arbitrary price for a market. In a perpetuals system the price is not one input among many, it is the input: it decides mark to market, who is liquidatable, what a liquidation pays out, and what ADL takes. An arbitrary price is therefore an arbitrary transfer between open positions and the liquidity pool. The protocol cannot detect this condition, because the absence of a reference feed is indistinguishable from the normal configuration of a token that never had one.

RECOMMENDATION

Make the presence of a reference feed a property of a listed market rather than an optional lookup. If a market is enabled for trading, require a non-zero price feed and revert otherwise, so removing the feed and continuing to trade becomes impossible rather than silent. Bound maxRefPriceDeviationFactor and maxPriceAge with hard constants in the contract so that the configurable range is a range a human would choose rather than the full width of a uint256. And emit a dedicated event whenever a token's price feed address changes, which is the minimum needed for anyone outside the team to notice that it did.

```
// ChainlinkPriceFeedUtils.sol:17-21
address priceFeedAddress = dataStore.getAddress(Keys.priceFeedKey(token));
if (priceFeedAddress == address(0)) {
return (false, 0);   // deviation check is then skipped entirely
}
```

GMX-02 · No SWC mapping (centralization)

Medium

DataStore accepts any key from any of 43 controllers and emits no events

DESCRIPTION

Every configuration value in GMX v2 lives in one DataStore keyed by a bytes32. The setters apply no restriction on which key may be written:

```
function setUint(bytes32 key, uint256 value) external onlyController returns (uint256) {
uintValues[key] = value;
return value;
}
```

setAddress and setBool are the same shape. The only gate is the CONTROLLER role, and the store does not care which controller is calling or which key is being written. A contract trusted to update funding rates is, at this layer, equally able to zero a price feed address.

Role membership was read from the live RoleStore on 22 October 2026. CONTROLLER is held by 43 addresses. The ones that could be identified are the protocol's own handlers, routers, factories and the Oracle itself, which is the expected architecture, but the count is the point: the blast radius of any single one of them is the entire configuration surface rather than the slice it needs.

DataStore emits no events. A configuration change is therefore invisible except as a state diff that someone has to know to look for. GMX does route changes through a Config contract that restricts what a CONFIG_KEEPER may set, and that restriction was not part of the verified bundle under review here, so this finding is deliberately scoped to what the DataStore itself enforces, which is nothing.

IMPACT

A compromise of any one of the 43 controller contracts is a compromise of every protocol parameter, not of that contract's own function. Because no event is emitted, the first signal available to an outside observer is the behaviour change itself, after it has taken effect.

RECOMMENDATION

Emit an event on every DataStore write carrying the key, the new value and msg.sender. The gas cost is small against what it buys, which is that every configuration change becomes indexable and alertable by anyone, not only by the team. Separately, consider narrowing CONTROLLER into per domain roles so that a handler can write the keys it owns and no others, which turns the blast radius of a single compromise from total into local.

GMX-03 · No SWC mapping (observability)

Medium

RoleStore emits no events for any role grant or revocation

DESCRIPTION

RoleStore contains no event declarations and no emit statements anywhere in the file. grantRole and revokeRole change authority through storage writes alone:

```
function _grantRole(address account, bytes32 roleKey) internal {
roles.add(roleKey);
roleMembers[roleKey].add(account);
```

```
roleCache[account][roleKey] = true;
}
```

There is also no restriction on which role may be granted. ROLE_ADMIN can grant any role to any address, CONTROLLER and ROLE_ADMIN among them. The only invariants enforced are on removal, that at least one ROLE_ADMIN and one TIMELOCK_MULTISIG must remain.

The mitigating fact is real and was verified on chain rather than assumed. ROLE_ADMIN is held by three addresses and all three are contracts, not keys: GovTimelockController at 0x4bd1cd..., ConfigTimelockController at 0x2dd99f..., and TimelockConfig at 0xe77069.... A grant therefore has to pass through a timelock, and the timelock emits its own events. That is a good answer to the question of who can change the rules, and better than most protocols of this size manage.

It is still worth recording, because it means the authority record of the system lives in the history of three other contracts rather than in the contract that holds the authority. Anyone auditing or monitoring GMX has to know that, and nothing in RoleStore says so.

IMPACT

Role changes cannot be indexed, subscribed to, or alerted on from RoleStore itself. A monitor built against the contract that actually holds the roles would see nothing at all. In the event that a role were ever granted by a path other than the timelocks, there would be no log of it.

RECOMMENDATION

Declare and emit RoleGranted and RoleRevoked events carrying the account, the role key and msg.sender, matching the convention of OpenZeppelin AccessControl. This costs one event per grant and makes the system's authority history self contained and independently monitorable.

GMX-04 · No SWC mapping (key management)

Low

CONFIG_KEEPER is held by five externally owned accounts

DESCRIPTION

Reading role membership from the live contract, CONFIG_KEEPER has six members. Five are externally owned accounts, meaning plain keys with no contract logic behind them:

0xf86ef7f0bb90a05a932781c5cb6eefe55e486107 EOA
0xf09d66cf7debcdebf965f1ac6527e1aa5d47a745 EOA
0xc5e038d696d9cb757ffdf53aa34e515d0e42f7cd EOA
0xb10e24b211d2321b12b21c52eac38ac6e00799cc EOA
0xd5f8b9ba4255b2f73b06f245fcca73d114d1d460 EOA
0x8d1d2e24ec641edc6a1ebe0f3ae7af0ebc573e0d GnosisSafeProxy

MARKET_KEEPER is held by five of the same six addresses. The severity is Low rather than higher because CONFIG_KEEPER is not CONTROLLER: a keeper acts through the Config contract, which restricts the keys it may set and which was not part of this verified bundle. What can be stated from what was reviewed is the key management fact, which is that five single keys sit in a privileged role and the sixth is the only one requiring more than one signature to use.

IMPACT

The practical ceiling on what a compromised CONFIG_KEEPER key can do is set by the Config contract's allowlist, not by anything in the contracts reviewed here. Any assessment of this role's risk depends on reading that allowlist, which is the recommendation below.

RECOMMENDATION

Confirm independently which keys CONFIG_KEEPER may set and whether any of them reach the oracle configuration described in GMX-01, since that is the question this finding cannot answer from the bundle under review. Separately, moving the remaining five keeper keys behind the same multisig already in the role would remove five single points of failure at no cost to operations.

GMX-05 · Not applicable

Informational

Arbitrary external calls are correctly sandboxed, recorded as a reference implementation

DESCRIPTION

ExchangeRouter exposes makeExternalCalls, which executes caller supplied calldata against caller supplied targets. That is normally the most dangerous function a router can have, and this repository already carries a High finding against a router where exactly this shape, combined with the router holding user ERC-20 allowances, turned every outstanding approval into something a single key could move.

GMX v2 does it the other way round. The router does not execute the calls. It forwards them to a separate ExternalHandler contract, so msg.sender for the arbitrary call is the handler and not the router, and the router's allowances are unreachable from the call. Tokens left behind in the handler are swept out by the refundTokens parameter in the same transaction.

The design is worth naming because the difference between the two implementations is not the presence of an arbitrary call. Both have one. The difference is which contract holds the user's approvals when it executes.

IMPACT

None. Recorded as a positive finding and as the correct pattern for any protocol that needs to compose with aggregators.

RECOMMENDATION

No change. Teams adding aggregator or swap composition to a router should execute arbitrary calls from a contract that holds no allowances, as is done here, rather than from the router that users approve.

GMX-06 · Not applicable

Informational

Auto deleveraging is gated on both a market flag and oracle freshness

DESCRIPTION

ADL is the mechanism that closes profitable positions when a pool cannot cover them, which makes it the path in a perpetuals system most able to take money from users who did nothing wrong. validateAdl refuses to proceed unless two conditions hold:

```
bool isAdlEnabled = AdlUtils.getIsAdlEnabled(dataStore, market, isLong);
if (!isAdlEnabled) { revert Errors.AdlNotEnabled(); }

uint256 latestAdlTime = AdlUtils.getLatestAdlTime(dataStore, market, isLong);
```

```
if (oracle.maxTimestamp() < latestAdlTime) {
revert Errors.OracleTimestampsAreSmallerThanRequired(oracle.maxTimestamp(), latestAdlTime);
}
```

The second condition is the one worth noting. It requires that the price being used is no older than the moment the ADL flag was raised, which closes the obvious abuse: a keeper cannot raise the flag and then deleverage against a price from before it. The flag is per market and per side rather than global, so enabling it on one market does not arm it everywhere.

IMPACT

None. Recorded as a positive finding. Note that it is a check on freshness rather than on value, so its strength still rests on the price itself being sound, which is the subject of GMX-01.

RECOMMENDATION

No change. Teams building liquidation or deleverage keepers should copy the pattern of binding the permitted price window to the moment the condition was recorded, rather than accepting any price that passes a staleness test.

Appendix · Metadata

Contract	GMX v2 (ExchangeRouter, RoleStore, DataStore, Oracle)
Address	0x69C527fC77291722b52649E45c838e41be8Bf5d5
Chain	Arbitrum
Compiler	v0.8.18+commit.87f61d96
Audit ID	gmx-v2-2026-10-22
Date	10/22/2026

Files analyzed

```
[dependency] @chainlink/contracts/src/v0.8/shared/interfaces/AggregatorInterface.sol
[dependency] @chainlink/contracts/src/v0.8/shared/interfaces/AggregatorV2V3Interface.sol
[dependency] @chainlink/contracts/src/v0.8/shared/interfaces/AggregatorV3Interface.sol
[dependency] @openzeppelin/contracts/security/ReentrancyGuard.sol
[dependency] @openzeppelin/contracts/token/ERC20/ERC20.sol
[dependency] @openzeppelin/contracts/token/ERC20/IERC20.sol
[dependency] @openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol
[dependency] @openzeppelin/contracts/token/ERC20/extensions/IERC20Permit.sol
[dependency] @openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol
[dependency] @openzeppelin/contracts/utils/Address.sol
[dependency] @openzeppelin/contracts/utils/Context.sol
[dependency] @openzeppelin/contracts/utils/math/Math.sol
[dependency] @openzeppelin/contracts/utils/math/SafeCast.sol
[dependency] @openzeppelin/contracts/utils/math/SafeMath.sol
[dependency] @openzeppelin/contracts/utils/math/SignedMath.sol
[dependency] @openzeppelin/contracts/utils/structs/EnumerableSet.sol
[user] contracts/adl/AdlUtils.sol
[user] contracts/bank/Bank.sol
[user] contracts/bank/StrictBank.sol
[user] contracts/callback/CallbackUtils.sol
[user] contracts/callback/IDepositCallbackReceiver.sol
[user] contracts/callback/IGasFeeCallbackReceiver.sol
[user] contracts/callback/IGlvDepositCallbackReceiver.sol
[user] contracts/callback/IOrderCallbackReceiver.sol
[user] contracts/callback/IShiftCallbackReceiver.sol
[user] contracts/callback/IWithdrawalCallbackReceiver.sol
[user] contracts/chain/ArbSys.sol
[user] contracts/chain/Chain.sol
[user] contracts/data/DataStore.sol
[user] contracts/data/Keys.sol
[user] contracts/deposit/Deposit.sol
[user] contracts/deposit/DepositEventUtils.sol
[user] contracts/deposit/DepositStoreUtils.sol
[user] contracts/deposit/DepositUtils.sol
[user] contracts/deposit/DepositVault.sol
```

[user] contracts/deposit/ExecuteDepositUtils.sol
[user] contracts/error/ErrorUtils.sol
[user] contracts/error/Errors.sol
[user] contracts/event/EventEmitter.sol
[user] contracts/event/EventUtils.sol
[user] contracts/exchange/IDepositHandler.sol
[user] contracts/exchange/IGlvHandler.sol
[user] contracts/exchange/IOrderHandler.sol
[user] contracts/exchange/IShiftHandler.sol
[user] contracts/exchange/IWithdrawalHandler.sol
[user] contracts/external/IExternalHandler.sol
[user] contracts/feature/FeatureUtils.sol
[user] contracts/fee/FeeUtils.sol
[user] contracts/gas/GasUtils.sol
[user] contracts/glv/Glv.sol
[user] contracts/glv/GlvDeposit.sol
[user] contracts/glv/GlvDepositEventUtils.sol
[user] contracts/glv/GlvDepositStoreUtils.sol
[user] contracts/glv/GlvDepositUtils.sol
[user] contracts/glv/GlvUtils.sol
[user] contracts/glv/GlvVault.sol
[user] contracts/market/Market.sol
[user] contracts/market/MarketEventUtils.sol
[user] contracts/market/MarketPoolValueInfo.sol
[user] contracts/market/MarketStoreUtils.sol
[user] contracts/market/MarketToken.sol
[user] contracts/market/MarketUtils.sol
[user] contracts/nonce/NonceUtils.sol
[user] contracts/oracle/ChainlinkPriceFeedUtils.sol
[user] contracts/oracle/IOracleProvider.sol
[user] contracts/oracle/IPriceFeed.sol
[user] contracts/oracle/Oracle.sol
[user] contracts/oracle/OracleStore.sol
[user] contracts/oracle/OracleUtils.sol
[user] contracts/order/BaseOrderUtils.sol
[user] contracts/order/IBaseOrderUtils.sol
[user] contracts/order/Order.sol
[user] contracts/order/OrderEventUtils.sol
[user] contracts/order/OrderStoreUtils.sol
[user] contracts/order/OrderVault.sol
[user] contracts/position/Position.sol
[user] contracts/position/PositionStoreUtils.sol
[user] contracts/position/PositionUtils.sol
[user] contracts/price/Price.sol
[user] contracts/pricing/ISwapPricingUtils.sol
[user] contracts/pricing/PositionPricingUtils.sol
[user] contracts/pricing/PricingUtils.sol
[user] contracts/pricing/SwapPricingUtils.sol

[user] contracts/referral/IReferralStorage.sol
[user] contracts/referral/ReferralEventUtils.sol
[user] contracts/referral/ReferralTier.sol
[user] contracts/referral/ReferralUtils.sol
[user] contracts/role/Role.sol
[user] contracts/role/RoleModule.sol
[user] contracts/role/RoleStore.sol
[user] contracts/router/BaseRouter.sol
[user] contracts/router/ExchangeRouter.sol
[user] contracts/router/IExchangeRouter.sol
[user] contracts/router/Router.sol
[user] contracts/shift/Shift.sol
[user] contracts/shift/ShiftEventUtils.sol
[user] contracts/shift/ShiftStoreUtils.sol
[user] contracts/shift/ShiftUtils.sol
[user] contracts/shift/ShiftVault.sol
[user] contracts/swap/SwapHandler.sol
[user] contracts/swap/SwapUtils.sol
[user] contracts/token/IWNT.sol
[user] contracts/token/TokenUtils.sol
[user] contracts/utils/AccountUtils.sol
[user] contracts/utils/Array.sol
[user] contracts/utils/Bits.sol
[user] contracts/utils/Calc.sol
[user] contracts/utils/Cast.sol
[user] contracts/utils/EnumerableValues.sol
[user] contracts/utils/PayableMulticall.sol
[user] contracts/utils/Precision.sol
[user] contracts/utils/Uint256Mask.sol
[user] contracts/withdrawal/ExecuteWithdrawalUtils.sol
[user] contracts/withdrawal/Withdrawal.sol
[user] contracts/withdrawal/WithdrawalEventUtils.sol
[user] contracts/withdrawal/WithdrawalStoreUtils.sol
[user] contracts/withdrawal/WithdrawalUtils.sol
[user] contracts/withdrawal/WithdrawalVault.sol
[dependency] prb-math/contracts/PRBMath.sol
[dependency] prb-math/contracts/PRBMathUD60x18.sol

MANUAL AUDIT UPGRADE

Need a deeper audit?

$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

Request Manual Audit ’

https://cryptohawking.com/audit/manual?ref=gmx-v2-2026-10-22

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

https://cryptohawking.com/services

WhatsApp (preferred): +92 322 4274236, https://wa.me/923224274236